

TrafficEase BD — Viva Preparation Guide

CSE 470: Software Engineering Project Architecture | Group 4

1. API & Key Configuration Matrix

A common trap in vivas is being asked where your API keys are stored. This table outlines every external API used in the project and where it is configured:

Service / API	Purpose	API Endpoint / Protocol	Key Required?
Google Maps Traffic Overlay	Renders live traffic density lines over the base map layer.	<code>https://{s}.google.com/vt/lyrs=m,traffic&x;={x}&y;={y}&z;={z}</code>	No API Key (Tile layer fetch)
Nominatim Geocoding Search	Converts typed Dhaka addresses to latitude/longitude.	<code>https://nominatim.openstreetmap.org/search</code>	No API Key (Public open API)
OSRM Routing Engine	Calculates fastest route path geometry between two points.	<code>https://router.project-osrm.org/route/v1/driving/...</code>	No API Key (Public demo server)
MongoDB Atlas / Database	Persistent storage for models (incidents, vehicles, etc).	<code>process.env.MONGO_URI</code> (backend/.env)	No API Key (Uses DB string)

CRITICAL VIVA Q&A:: 'Where is your Google Maps API key?'
Answer: 'We integrated Google Maps traffic overlays directly into React-Leaflet via a custom `TileLayer` pointing to Google's public map server. Since it renders direct map tiles inside the Leaflet viewport rather than loading the premium Google Maps JavaScript SDK, it does not require an API key. Similarly, for geocoding and routing, we leveraged public keyless endpoints from Nominatim and OSRM.'

2. Deep Dive: The 11 Core Features & Codes

Feature 1: Google Maps Traffic Overlay

Code Location: `frontend/src/pages/LiveMap.js` (L354-356)
`frontend/src/pages/ReportIncident.js` (L263-265)

How it works: Leaflet's `TileLayer` queries Google's public tile server layout using `lyrs=m,traffic` and standard coordinates variables, loading colored speed markers natively.

Viva Question: 'Why Leaflet instead of Google Maps API SDK?'
Answer: Leaflet is free, lightweight, open-source, and allows us to easily place custom SVG markers, popups, and route layers on top of various base maps.

Feature 2: Nominatim Geocoding Search Engine

Code Location: `frontend/src/pages/LiveMap.js` (L106-120)
`frontend/src/pages/ReportIncident.js` (L85-98)

How it works: Sends an asynchronous fetch query to Nominatim's public endpoint. Extracts coordinates from the first JSON result and pans the map context using Leaflet's `setView()`.

Viva Question: 'What happens if Nominatim fails or returns nothing?'
Answer: We catch fetch exceptions and check for empty results, warning the user with a UI toast/error message instead of crashing the page.

Feature 3: Click-to-Pin Location Picker

Code Location: `frontend/src/pages/ReportIncident.js` (L27-40) [`LocationPicker` component]

How it works: Uses Leaflet's `useMapEvents` hook inside the `MapContainer` to intercept user click events on the canvas, extracting `latlng` parameters and feeding them into the state form.

Viva Question: 'Why could you not use a standard React `onClick` event?'
Answer: Standard DOM `onClick` events only provide page pixels. We must use Leaflet's coordinate projection system (via `useMapEvents`) to translate pixels to geographic latitude and longitude.

Feature 4: Multimodal Travel Duration & ETA Estimator

Code Location: frontend/src/services/trafficCalculations.js (L28-35) [etaComparison]
frontend/src/pages/LiveTraffic.js (L950-977) [case 26]

How it works: Reads distance inputs and calculates estimated trip durations for private cars, buses, and metro feeders using dynamic simulation multipliers.

Viva Question: 'Are these travel times connected to real-world traffic sensors?'

Answer: They are simulated models for operational demonstration. In a live system, the multipliers would be replaced with real-time API telemetry datasets.

Feature 5: Weather & Flood Sensor Risk Estimators

Code Location: frontend/src/services/trafficCalculations.js (L26)
frontend/src/pages/LiveTraffic.js (L514-570) [case 11 & case 12]

How it works: Computes relative road hazard index from rain precipitation and triggers alerts when waterlogged heights exceed passenger car clearance (18 inches).

Viva Question: 'What is the operational purpose of this simulator?'

Answer: It simulates central control warnings. When monsoon waterlogging renders a road impassable, authorities get a warning to dispatch WASA pumps and alert drivers.

Feature 6: Real-time Congestion Trend Chart

Code Location: frontend/src/pages/LiveTraffic.js (L151-166) [SVG path generation]
frontend/src/pages/LiveTraffic.js (L1173-1186) [Rendering]

How it works: Calculates real-time SVG paths (M/L line commands) inside a React useMemo hook, scaling historical congestion points to fit into a 300x80 canvas box.

Viva Question: 'Why calculate SVG paths in React instead of using standard chart libraries?'

Answer: Direct SVG rendering uses zero dependencies, compiles to minimal code size, renders instantly, and is highly customizable with plain CSS styling.

Feature 7: Incident Purging Handler

Code Location: backend/controllers/incidentController.js (L116-131)
frontend/src/pages/Dashboard.js (L57-66)

How it works: Sends an authorized DELETE HTTP request. The backend middleware decodes the sender's JWT to verify Admin/Authority role clearance and removes the item from the DB.

Viva Question: 'How is authentication handled on this delete route?'

Answer: We use a JWT validation middleware ('protect') that reads the bearer token from request headers, decoding user information and verifying permission policies before processing the delete query.

Feature 8: Adaptive Signal Timing Calculator

Code Location: frontend/src/services/trafficCalculations.js (L20-24) [adaptiveSignalPlan]
frontend/src/pages/LiveTraffic.js (L349-376) [case 5]

How it works: Takes intersection vehicle load percentage and returns adjusted green phase seconds (30s baseline up to 90s maximum) and estimated throughput improvements.

Viva Question: 'What problem does adaptive signaling solve?'

Answer: Pre-timed traffic lights create massive delays on empty streets. Adaptive signaling recalculates green times dynamically to prioritize congested lanes.

Feature 9: Corridor Load Indicators

Code Location: frontend/src/pages/LiveTraffic.js (L1224-1246)

How it works: Loops over monitored Dhaka highways to display speeds, delay indexes, and colored bars representing relative congestion load.

Viva Question: 'How does this layout hook into the detailed view?'

Answer: Clicking any corridor row changes the `selectedCorridor` state, dynamically updating the adjacent detail panels with bottleneck causes and recommendations.

Feature 10: Signal Failure Controller Alerts

Code Location: frontend/src/pages/LiveTraffic.js (L378-414) [case 6]

How it works: Renders toggles for intersection controllers. Activating any switch generates warning messages and flags the database context with error signals.

Viva Question: 'Where does this event write logs?'

Answer: Toggling a warning fires a dispatch audit event, which gets appended directly to the session audit console visible at the bottom of the simulator panel.

Feature 11: Interactive Parking Spot Allocator

Code Location: frontend/src/pages/LiveTraffic.js (L719-754) [case 18]

How it works: Renders a Kemal Atatürk parking lot grid. Users click slots to reserve/free spaces, which dynamically calculates remaining available parking lots.

Viva Question: 'How does the UI state update on click?'

Answer: Clicking a slot toggles a boolean flag inside an array in React state, which changes the styling (green for free, red for booked) and re-evaluates the array count.

3. General Architectural Questions

Q: What architecture design pattern does the project implement?

It follows a decoupled Model-View-Controller (MVC) architecture. React views handle UI presentation and request actions; Express routes and controllers handle backend routing and endpoints; Mongoose schemas manage database collections.

Q: What happens if MongoDB goes offline during a live presentation?

We built a robust resiliency fallback mechanism. If the database connection fails, controllers automatically shift data storage to an in-memory mock database (`backend/data/mockDatabase.js`). This ensures the application remains functional for classroom demonstrations.

Q: How do you secure user passwords in the database?

We hash user passwords before storing them in MongoDB using `bcryptjs`. In the User Model, we implement a pre-save schema hook that triggers whenever password fields are created or modified, ensuring raw passwords never enter our storage layer.

Q: What is the role of the backend 'Services' layer?

To keep controllers thin, we separate concerns. Calculations, policies, and token generators reside in `'backend/services/'` (e.g. `tokenService.js`, `authPolicy.js`), keeping routes focused only on parsing requests and returning JSON responses.

4. Presentation Walkthrough Script

Use this structured script to guide your screen-sharing presentation in front of the examiners:

Step 1: Introduction (Landing Screen): Point out the landing page. Mention the tech stack (React, Node, Express, MongoDB). Highlight role-based authorization: Commuter, Driver, Authority, Admin.

Step 2: Dashboard & Incident Purging: Log in as an Administrator/Authority. Navigate to the Dashboard. Locate an incident report and click **Delete** (Feature 7). Mention that this executes a DELETE request on the database, validated behind JWT middleware policies.

Step 3: GIS Map & Geocoding: Navigate to the Map view. Show the Google Traffic color tracks (Feature 1). Type a location in the search bar and pan the map (Feature 2). Open the 'Report Incident' page and click on the map to pin coords (Feature 3), explaining how `useMapEvents` bridges Leaflet and forms.

Step 4: Operations Simulator: Open the Live Traffic operations hub. Run through the sliders: Show the SVG Trend Chart (Feature 6), click corridor rows to watch bars update (Feature 9), drag the demand slider for adaptive green timing (Feature 8), slide distance for Multimodal ETA comparison (Feature 4), adjust rain inputs for waterlogging alerts (Feature 5), and click parking grid buttons to allocate slots (Feature 11). Focus on the audit trail log updating in real-time at the bottom.